

```
# Backend Core Infrastructure Reference
# Production E-Commerce Platform – Django + DRF + Celery

> **Purpose of this document**
> This file is the complete reference for the core infras
> Django backend. Any AI model or developer onboarding to
> this entire document before writing a single line of co
> convention, and constraint described here is already im
> production use. Do not invent alternatives. Do not gues
> file if you need something not covered here.
```

Table of Contents

1. Project Overview
2. Technology Stack
3. Application Registry
4. Six-Layer Architecture
5. API Response Envelope – The Contract
6. Exception Hierarchy
7. Global Exception Handler
8. BaseAPIView – All Views Inherit This
9. APIResponseMixin
10. Error Codes Registry
11. Two-Level Cache System
12. Pagination System
13. Serializer Fields and Mixins
14. Permission Classes
15. Middleware
16. Authentication and Token Strategy
17. Logging System
18. Settings Reference
19. Critical Rules – Never Violate
20. How to Add a New Endpoint – Checklist
21. Appendix A – Response Examples by Scenario
22. Appendix B – Import Reference

1. Project Overview

This is a production-scale e-commerce backend built for t
It follows Daraz-style patterns: verified purchases, COD
handling, local courier integrations (PostEx, TCS, Leopard
(Return to Origin) tracking.

Total Apps: 19 (3 core/infrastructure + 16 domain)
Total Domain Models: 29+

The backend is a REST API consumed by a React frontend. A
via Postman. The architecture enforces strict consistency
returns the identical envelope shape regardless of succes

2. Technology Stack

Layer	Technology
Web Framework	Django 4.x
API Framework	Django REST Framework (D
Async Task Queue	Celery
Message Broker / Cache	Redis (port 6380 – NOT d
Database	PostgreSQL
Authentication	SimpleJWT (JWT tokens)
API Documentation	drf-spectacular (OpenAPI
Query Layer	GraphQL via graphene-dja
Static Files	WhiteNoise
Error Monitoring	Sentry SDK
Environment Config	django-environ
Dev Tooling	django-debug-toolbar, dj

3. Application Registry

Infrastructure Apps (no domain logic)

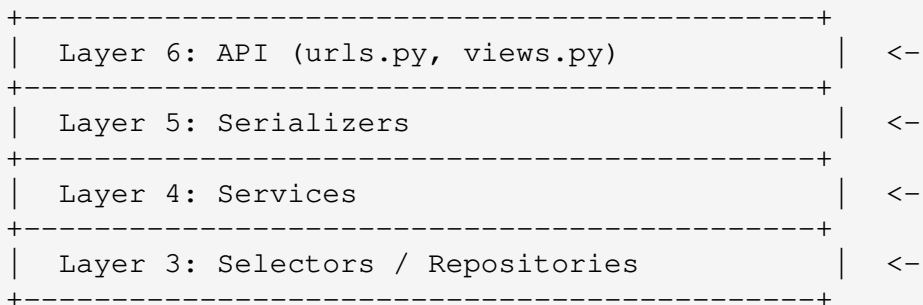
- apps.core - Base views, exceptions, cache, pagination, middleware, error codes, logging framework
- apps.common - Shared choices (Role, etc.), shared utilities

Domain Apps

- apps.accounts - Auth, JWT, profiles, addresses
- apps.products - Catalog, variants, attributes, filters
- apps.cart - Cart and cart items
- apps.coupons - Coupon engine
- apps.orders - Order creation, state machine, order tracking
- apps.payments - Payment gateway integrations
- apps.reviews - Verified-purchase reviews, voting
- apps.contact - Customer support messages and responses
- apps.wishlist - Wishlist items
- apps.tax - Tax calculation
- apps.returns - Return requests
- apps.recomendations - Interaction logging, personalization
- apps.notifications - Push / email notifications
- apps.logistics - Shipments, courier webhooks, COGS
- apps.analytics - Daily snapshots, courier performance

4. Six-Layer Architecture

Every domain app follows this strict layer separation. Imports must only flow downward. Upper layers import lower layers.



	Layer 2: Models		<-
+	-----	+	
	Layer 1: Core / Common		<-
+	-----	+	

Rules:

- Views call services, never query models directly
- Services raise DomainError or InfrastructureError,
- Serializers validate input, they do not contain bus
- Models contain field definitions and simple model-l
- The global exception handler converts all exception

5. API Response Envelope – The Contract

CRITICAL: This envelope shape is returned by EVERY endpoint without exception. The frontend relies on this. Never bre

```

{
  "success": true,
  "message": "Human readable summary",
  "data":    {},
  "errors":  null,
  "meta": {
    "request_id": "uuid-string"
  }
}

```

Field Rules

Field	Success (2xx)	Er
-----	-----	----
success	true	fa
message	Confirmation string	Sa
data	Payload object/array	nu
errors	null (always)	Er
meta	Always present, contains request_id	Al

Success Response – Full Example

```

{
  "success": true,
  "message": "Account created successfully. Please",
  "data": {
    "email": "user@example.com"
  },
  "errors": null,
  "meta": {
    "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234"
  }
}

```

Error Response – Full Example

```

{
  "success": false,
  "message": "Registration failed.",
  "data": null,
  "errors": {
    "code": "validation_error",
    "fields": {
      "email": {
        "message": "Enter a valid email address",
        "code": "invalid_email_format"
      },
      "password": {
        "message": "Password must be at least 8 characters",
        "code": "min_length"
      }
    },
    "non_fields": null
  },
  "meta": {
    "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234"
  }
}

```

The errors Object – Complete Structure

```

"errors": {
  "code": "top_level_error_code",
  "fields": {
    "<field_name>": {
      "message": "Human readable error for this",
      "code": "machine_readable_code"
    }
  },
  "non_fields": {
    "category": "validation | domain | system | u",
    "message": "Human readable error",
    "code": "machine_readable_code",
    "extra": {} | null
  }
}

```

fields – When Populated

- Serializer field-level validation failures only
- Each key = field name that failed
- Only the FIRST error per field is surfaced (DRF ret
- Multiple field errors returned simultaneously across

non_fields – Four Categories

Category	When Used
validation	DRF non_field_errors, auth failures, pe
domain	Business rule violation (raised as Doma
system	Infrastructure failure (raised as Infra
unexpected	Unhandled exception – always triggers m

HTTP Status to errors.code Mapping

HTTP Status	errors.code
400	validation_error
401	authentication_error
403	permission_error
404	not_found

405		method_not_allowed
409		conflict_error
429		rate_limit_exceeded
500		server_error
503		server_error

NOTE: 409 maps to conflict_error, NOT validation_error
 Rationale: validation_error means malformed data.
 409 means data was valid but current state refuses the request
 The frontend branches on the top-level code – conflates

HTTP Status Default Messages (safe, shown to users)

HTTP Status		Message
400		"Invalid request data."
401		"Authentication required."
403		"You do not have permission to perform"
404		"The requested resource was not found."
405		"Method not allowed."
409		"This action conflicts with the current"
429		"Too many requests. Please slow down."
500		"An unexpected error occurred. Please try"
503		"The service is temporarily unavailable"

Paginated Response – Meta Structure

When an endpoint returns a paginated list, the meta field

```
{
  "success": true,
  "message": "...",
  "data": [...],
  "errors": null,
  "meta": {
    "request_id": "...",
    "page": 1,
    "page_size": 20,
    "total": 540,
    "total_pages": 27,
```

```

        "showing_from": 1,
        "showing_to": 20,
        "has_next": true,
        "has_previous": false,
        "source": "l2_redis",
        "sort": "newest",
        "elapsed_ms": 12.4
    }
}

```

source, sort, and elapsed_ms are optional.
 null values are excluded from meta entirely – do not

6. Exception Hierarchy

File: apps/core/exceptions.py

CRITICAL RULE: This file has ZERO imports from Django HT
 apps.core.api. It is safe to import from any layer includ
 and management commands.

```

BaseAppError(Exception)
├── DomainError          → Business rule violations →
└── InfrastructureError  → Infrastructure failures →

```

BaseAppError

Never raise directly. Exists for catch-all handlers.

```

BaseAppError(
    message: str,                                # human-reada
    *,
    code: str,                                    # machine-r
    client_extra: dict | None = None,            # forwarded
    internal: dict | None = None,                # NEVER for
)
.status_code = 0  (sentinel, overridden by subclasses

```


DomainError

Use when the domain says NO to a valid request.

Not a malformed request – the data was fine but business

```
DomainError(
    message: str,
    *,
    code: str = "domain_error",
    status_code: int = 400,      # MUST be 400 or 409
    client_extra: dict | None = None,
    internal: dict | None = None,
)
ALLOWED_STATUS_CODES = frozenset({400, 409})
```

When to use 400 vs 409:

400 – Generic domain rejection: "Cart is empty", "Cou

409 – State conflict: "Order already cancelled", "Ema

Examples:

```
# 400 – generic rejection
raise DomainError(
    "Your cart is empty. Add items before checking out",
    code="cart_empty",
)
```

```
# 409 – state conflict
raise DomainError(
    "This order has already been cancelled.",
    code="order_already_cancelled",
    status_code=409,
)
```

```
# 409 with client_extra (forwarded to response) and i
raise DomainError(
    "Coupon has already been used.",
    code="coupon_already_used",
    status_code=409,
    client_extra={"applied_on": "2024-01-15"},
)
```

```
        internal={"coupon_id": 99, "used_by_user_id": 12}
    )
```

InfrastructureError

Use when an external system or internal subsystem is temporarily unavailable. Always returns 503. Not configurable.

IMPORTANT: Named InfrastructureError NOT SystemError to avoid shadowing Python's built-in SystemError, which the interpreter and CPython use for internal VM-level failures. Shadowing it causes silent failures in third-party libraries that catch SystemError

```
InfrastructureError(
    message: str,
    *,
    code: str = "infrastructure_error",
    notify: bool = True,      # True = alert Sentry/PagerDuty
                              # False = WARNING log only
    client_extra: dict | None = None,
    internal: dict | None = None,
)
.status_code = 503  (class attribute, not overridable)
```

Examples:

```
# Unexpected failure – alert on-call
raise InfrastructureError(
    "Payment gateway is temporarily unavailable. Please try again later.",
    code="payment_gateway_timeout",
    client_extra={"retry_after": 30},
    internal={"gateway": "stripe", "http_status": 504}
)
```

```
# Expected, handled outage – fallback in place, no pager
raise InfrastructureError(
    "Recommendations unavailable. Showing defaults.",
    code="recommendation_service_down",
    notify=False,
    internal={"service": "rec-engine", "reason": "circuit breaker"}
)
```

)

7. Global Exception Handler

File: apps/core/api/exceptions.py

Registered in settings: "EXCEPTION_HANDLER": "apps.core.a

THE SINGLE MOST IMPORTANT ARCHITECTURAL INVARIANT:

All logging lives in custom_exception_handler and ONLY th

Builder functions are pure – input in, dict out, zero sid

This guarantees exactly ONE log entry and ONE Sentry even

SECURITY INVARIANT:

exc.internal is NEVER passed to any builder function and

any response. It is only written to the logger inside cus

Decision Tree (evaluated in order)

Exception raised

|— isinstance(exc, DomainError)?

| status = exc.status_code (400 or 409)

| non_fields.category = "domain"

| monitor alert = NO

| log = WARNING, no traceback

| exc.internal = logged at DEBUG before buil

| response body = _non_fields_domain(exc)

|— isinstance(exc, InfrastructureError)?

| status = 503

| non_fields.category = "system"

| if notify=True → _notify_monitors(exc) + ERR

| if notify=False → WARNING log, no traceback,

| response body = _non_fields_infrastructure(

|— DRF-recognised exception? (exception_handler retu

| status = original DRF status code pr

| non_fields.category = "validation"

```

|         if status >= 500 → _notify_monitors + ERROR log
|         if status < 500 → WARNING log, no traceback
|         response body    = _format_drf_errors(response)
|
└─ Unhandled exception (exception_handler returns None)
    status                = 500
    non_fields.category = "unexpected"
    always                → _notify_monitors + ERROR log
    DEBUG=True           → raw exc message in response
    DEBUG=False          → safe generic message only ( )

```

Monitor Registry

```

from apps.core.api.exceptions import register_monitor

# Register Sentry or any monitor – called once at startup
register_monitor(lambda exc: capture_exception(exc))

```

Monitors ARE called for:

- InfrastructureError where notify=True
- DRF 5xx responses
- Unhandled exceptions

Monitors are NOT called for:

- DomainError (expected business logic)
- DRF 4xx (client errors, not our defects)
- InfrastructureError where notify=False

Key Helper Functions (all pure – zero side effects, zero dependencies)

```

_format_drf_errors(data, status_code) -> dict
Normalizes DRF error data into the standard error dict
Called by:
- custom_exception_handler for DRF exceptions
- BaseAPIView.error_response() for manual responses
- BaseAPIView.not_found_response(), unauthorized responses
Handles three DRF shapes:
dict    → {"email": [ErrorDetail], "non_field_errors": [ErrorDetail]}
list    → [ErrorDetail("Authentication credentials were not provided."),
            ErrorDetail("Not found.")]
str      → "Not found."

```

```

_status_to_error_code(status_code) -> str
    Maps HTTP status to top-level errors.code string.
    409 maps to conflict_error, not validation_error.

_status_to_message(status_code) -> str
    Maps HTTP status to safe default human message.

_non_fields_domain(exc: DomainError) -> dict
    Builds domain non_fields block. Pure. No logging.

_non_fields_infrastructure(exc: InfrastructureError)
    Builds system non_fields block. Pure. No logging.

_non_fields_unexpected(exc, *, status_code) -> dict
    Builds unexpected non_fields block. DEBUG shows r
    Production always shows safe generic message.

```

8. BaseAPIView – All Views Inherit This

File: apps/core/api/views.py

```

from apps.core.api.views import BaseAPIView

class MyView(BaseAPIView):
    ...

```

RULE: Every view in the project inherits from BaseAPIView
 Never use APIView or GenericAPIView directly.

BaseAPIView inherits from: APIResponseMixin, GenericAPIVi
 Default: permission_classes = [AllowAny] – override per

Response Helper Decision Guide

Situation	Helper to Use
Successful GET, list, or action	self.success

Resource just created (POST)		self.created
Serializer field errors		self.error_r
DomainError caught directly in view		self.domain_
Resource does not exist		self.not_fou
No auth credentials		self.unautho
Auth present but access denied		self.forbidd

PREFERRED PATTERN: Raise DomainError / Infrastructure layer and let custom_exception_handler handle automatically. Only use domain_error_response() when a view constructs directly without a service layer in between (rare).

Full Method Signatures

— Success —————

```
def success_response(
    self,
    *,
    data: Any = None,
    message: str = "Request successful",
    status_code: int = 200,
    meta: dict | None = None,
) -> Response
```

```
def created_response(
    self,
    *,
    data: Any = None,
    message: str = "Resource created successfully",
    meta: dict | None = None,
) -> Response
```

— Error —————

```
def error_response(
    self,
    *,
    message: str = "Request failed",
    errors: Any = None, # serializer.errors,
```

```

        status_code: int = 400,
    ) -> Response
    # Internally calls _format_drf_errors(errors, status_code)
    # Do NOT use for domain errors – use domain_error_response
    # so frontend receives category="domain" not category="validation_error"

    def domain_error_response(
        self,
        *,
        exc: DomainError,
    ) -> Response
    # Produces category="domain", correct status, client_error

    def not_found_response(
        self,
        *,
        message: str = "Resource not found",
    ) -> Response
    # Returns 404, errors.code="not_found", category="validation_error"

    def unauthorized_response(
        self,
        *,
        message: str = "Authentication required",
    ) -> Response
    # Returns 401, errors.code="authentication_error", category="validation_error"

    def forbidden_response(
        self,
        *,
        message: str = "You do not have permission to perform this action",
    ) -> Response
    # Returns 403, errors.code="permission_error", category="validation_error"

### transform_payload() – Request ID Injection

BaseAPIView overrides APIResponseMixin.transform_payload() to inject the
request_id into every response meta automatically.

def transform_payload(self, payload: dict) -> dict:

```

```

    request_id = getattr(self.request, "id", None)
    if request_id:
        if payload.get("meta") is None:
            payload["meta"] = {}
        payload["meta"]["request_id"] = request_id
    return payload

```

request.id is set by RequestIDMiddleware.

If middleware is absent (e.g. unit tests), this is a

9. APIResponseMixin

File: apps/core/api/mixins.py

BaseAPIView inherits from this. You do not use this direc

```

class APIResponseMixin:
    MESSAGE_KEY: str = "message"
    ERRORS_KEY: str = "errors"
    DATA_KEY: str = "data"
    SUCCESS_KEY: str = "success"
    META_KEY: str = "meta"

    def build_response(
        self,
        *,
        data: Any = None,
        message: str | None = None,
        errors: Any = None,
        status_code: int = 200,
        meta: dict | None = None,
        headers: dict | None = None,
    ) -> Response

```

```

    def transform_payload(self, payload: dict) -> dict:
        return payload # no-op in mixin, overridden in

```

The build_response() method:

1. Assembles the 5-key envelope (success, message, data, headers, status_code)
2. Calls self.transform_payload(payload) – injects response object
3. Creates Response(payload, status=status_code)
4. Applies any headers dict to the response object

10. Error Codes Registry

File: apps/core/error_codes.py

```
from apps.core.error_codes import ErrorCode
```

All machine-readable error codes live here.

No string literals at raise sites – always use ErrorCode
These are a PUBLIC API. The React frontend switches on them.
Changing a value string is a BREAKING CHANGE.

```
# — Authentication —————
ErrorCode.INVALID_CREDENTIALS          = "invalid_credentials"
ErrorCode.EMAIL_NOT_VERIFIED           = "email_not_verified"
ErrorCode.ACCOUNT_DISABLED             = "account_disabled"
ErrorCode.ACCOUNT_INACTIVE            = "account_inactive"
ErrorCode.SESSION_EXPIRED              = "session_expired"
ErrorCode.TOKEN_INVALID               = "token_invalid"
ErrorCode.TOKEN_EXPIRED               = "token_expired"

# — OTP —————
ErrorCode.OTP_RESEND_COOLDOWN          = "otp_resend_cooldown"
ErrorCode.OTP_NOT_FOUND               = "otp_not_found"
ErrorCode.OTP_EXPIRED                 = "otp_expired"
ErrorCode.OTP_INVALID                 = "otp_invalid"
ErrorCode.OTP_ATTEMPTS_EXCEEDED       = "otp_attempts_exceeded"

# — Verified Session —————
ErrorCode.VERIFICATION_SESSION_INVALID = "verification_session_invalid"
ErrorCode.VERIFICATION_SESSION_EXPIRED = "verification_session_expired"

# — Social Auth —————
ErrorCode.UNSUPPORTED_AUTH_PROVIDER   = "unsupported_auth_provider"
```

```

ErrorCode.INVALID_SOCIAL_TOKEN           = "invalid_soc
ErrorCode.AUTH_PROVIDER_UNREACHABLE      = "auth_provid
ErrorCode.SOCIAL_EMAIL_NOT_VERIFIED      = "social_email
ErrorCode.SOCIAL_EMAIL_MISSING           = "social_email
# NOTE: ACCOUNT_INACTIVE appears in both Auth and Soc

# — Registration / Email —————
ErrorCode.EMAIL_ALREADY_EXISTS           = "email_already
ErrorCode.EMAIL_INVALID_FORMAT           = "email_invalid

# — User / Profile —————
ErrorCode.INVALID_FULL_NAME              = "invalid_full_name
ErrorCode.INVALID_PHONE                  = "invalid_phone
ErrorCode.INVALID_DATE_OF_BIRTH          = "invalid_date_of_birth
ErrorCode.INVALID_ADDRESS                = "invalid_address

# — Password —————
ErrorCode.PASSWORD_TOO_WEAK              = "password_too_weak
ErrorCode.PASSWORD_MISMATCH              = "password_mismatch
ErrorCode.PASSWORD_SAME_AS_OLD           = "password_same_as_old

# — File Upload —————
ErrorCode.INVALID_IMAGE_SIZE             = "invalid_image_size
ErrorCode.INVALID_IMAGE_TYPE             = "invalid_image_type

# — Top-level Category Codes (go into errors.code no
# These describe WHAT KIND of error occurred, not the
ErrorCode.VALIDATION_ERROR               = "validation_error
ErrorCode.AUTHENTICATION_ERROR           = "authentication_error
ErrorCode.PERMISSION_ERROR               = "permission_error
ErrorCode.NOT_FOUND                      = "not_found
ErrorCode.RATE_LIMIT_EXCEEDED            = "rate_limit_exceeded
ErrorCode.SERVER_ERROR                   = "server_error
ErrorCode.METHOD_NOT_ALLOWED            = "method_not_allowed
ErrorCode.CONFLICT_ERROR                 = "conflict_error

```

When adding a new code:

1. Add constant to ErrorCode in apps/core/error_codes
2. Add a comment indicating which app/domain owns it
3. Never delete – deprecate with a comment instead

4. Inform the frontend team – these are a public cont

11. Two-Level Cache System

File: apps/core/cache.py

Singleton: from apps.core.cache import two_level_cache

Architecture

Request

|

v

L1: LocMemCache (caches["local"])
per-process, ultra-fast, 60s TTL, max 500 entries
miss?

|

v

L2: Redis (caches["default"])
shared across all workers, 300s TTL
miss?

|

v

Database (via builder_fn passed to get_or_set)

Key Concepts

Cache Versioning:

All keys prefixed with v{CACHE_VERSION}:

Currently CACHE_VERSION = 1

To bust entire cache on deploy: increment CACHE_V

Sentinel Encoding:

All values stored as {"__v": value}

Distinguishes genuine cache miss (backend returns
a cached falsy value (None, False, [], 0).

Without this, cached None looks like a miss → unn

Stampede Protection:

get_or_set() uses cache.add() (atomic SET NX) as
Only ONE worker rebuilds cache on cold miss.
All other concurrent workers wait up to _lock_max
If Redis unavailable: _acquire_lock fails open –
without stampede protection rather than crashing.

L1 Deep Copy:

L1 get() returns copy.deepcopy(value).
Without this, callers mutating returned object would
be shared in-process cache for all subsequent requests.

Fault Tolerance:

L1 and L2 failures are caught, logged as WARNING,
Cache failures degrade gracefully – they never crash.

Public API

```
from apps.core.cache import two_level_cache

# — Cache-aside (recommended pattern) —————
data, source = two_level_cache.get_or_set(
    "products:list:page1",
    lambda: MySerializer(MyModel.objects.all(), many=True),
    lock=True,                # default True – set False in tests
    l1_timeout=60,            # optional override of instance l1_timeout
    l2_timeout=300,           # optional override of instance l2_timeout
)
# source: "l1_memory" | "l2_redis" | "database"

# — Manual get —————
data, source = two_level_cache.get("products:list:page1")
if source == "miss":
    data = build_data()
    two_level_cache.set("products:list:page1", data)

# — Set —————
two_level_cache.set(
    "my_key",
    value,
    l1_timeout=30,            # optional override of instance l1_timeout
```

```

        l2_timeout=120,          # optional
    )

# — Delete (prefix-based, SCAN safe – never KEYS *)
two_level_cache.delete("products:")
# L2: SCAN for *v1:products:* pattern
# L1: full clear (cheap – rebuilds on next request)

```

Return Values

get() and get_or_set() both return (value, source):

source	Meaning
-----	-----
"l1_memory"	Served from in-process LocMemCache
"l2_redis"	Served from Redis; L1 backfilled au
"miss"	Not in either level – caller must r
"database"	Cache was cold, builder_fn() called

Pass source to build_pagination_meta() to expose cache response meta.source field.

Instance Defaults

```
two_level_cache = TwoLevelCache(l1_timeout=60, l2_timeout=120)
```

Override per-call using l1_timeout / l2_timeout keywords.
Different timeouts for different data profiles:

```

    Category list (rarely changes):  l2_timeout=3600
    Search results (change often):   l2_timeout=60
    User-specific data:               scope key to use

```

Cache Key Conventions

```

products:list:{filter_hash}
products:detail:{slug}
categories:tree
brands:list
cart:{user_id}
orders:list:{user_id}:page:{page}

```

12. Pagination System

File: apps/core/paginations.py

Two functions used together in every paginated view.

```
### get_pagination_params(request, default_page_size=20,

    from apps.core.paginations import get_pagination_params

    params = get_pagination_params(request)
    # Returns PaginationParams(page, page_size, is_valid)
    # PaginationParams is a NamedTuple – access as params

    if not params.is_valid:
        return self.error_response(message="Invalid pagination")
```

Behavior:

- Never raises exceptions – returns `is_valid=False` on
- Logs `WARNING` on invalid params (visible in `apps.core`)
- `page` is always `>= 1`
- `page_size` is always between 1 and `max_page_size`
- If `page` is valid but `page_size` is not: keeps valid
- Query params: `?page=2&page_size=12`

Default limits:

```
    default_page_size=20 – returned when page_size not in query
    max_page_size=48     – hard ceiling regardless of what is requested
```

Override per endpoint:

```
    params = get_pagination_params(request, default_page_size=10)
```

```
### build_pagination_meta(page, page_size, total, *, source_url=None)

    meta = build_pagination_meta(
        params.page,
        params.page_size,
```

```

        total_count,
        source=cache_source,      # "l1_memory" | "l2_redis"
        sort=sort_param,         # "newest" | "price_asc"
        elapsed_ms=12.4,         # optional timing
    )

```

Returns:

```

{
    "page":          1,
    "page_size":     20,
    "total":         540,
    "total_pages":   27,
    "showing_from":  1,
    "showing_to":    20,
    "has_next":      True,
    "has_previous":  False,
    # Optional – only included if passed as non-None:
    "source":        "l2_redis",
    "sort":          "newest",
    "elapsed_ms":    12.4,
}

```

clamp_page=True (default):

If page overshoots total_pages, it is clamped down to total_pages
 Prevents showing_from/showing_to from producing garbage

Complete Paginated View Pattern

```

import time
from apps.core.paginations import get_pagination_params
from apps.core.cache import two_level_cache

class ProductListAPIView(BaseAPIView):
    def get(self, request):
        params = get_pagination_params(request, default_params)
        if not params.is_valid:
            return self.error_response(message="Invalid parameters")

        sort = request.query_params.get("sort", "newest")
        cache_key = f"products:list:{sort}:page:{params.page}"

```

```

t0 = time.monotonic()

data, source = two_level_cache.get_or_set(
    cache_key,
    lambda: self._build_product_data(params,
)

elapsed_ms = (time.monotonic() - t0) * 1000

meta = build_pagination_meta(
    params.page,
    params.page_size,
    data["total"],
    source=source,
    sort=sort,
    elapsed_ms=round(elapsed_ms, 2),
)

return self.success_response(
    data=data["results"],
    message="Products retrieved successfully.",
    meta=meta,
)

```

13. Serializer Fields and Mixins

Files: apps/core/fields.py, apps/core/mixins.py

CustomDateTimeField

```

from apps.core.fields import CustomDateTimeField

class MySerializer(serializers.ModelSerializer):
    some_date = CustomDateTimeField()

```

Defaults:

```
format=None
```



```

input_formats=['iso-8601']
default_timezone=timezone.get_current_timezone()
read_only=True

### TimestampFieldsMixin

from apps.core.mixins import TimestampFieldsMixin

class MySerializer(TimestampFieldsMixin, serializers.
    class Meta:
        model = MyModel
        fields = [..., "created_at", "updated_at"]

Automatically adds created_at and updated_at as CustomDat
ISO-8601, read-only, timezone-aware.
Use this mixin on EVERY serializer that exposes timestamp
Do NOT redefine these fields manually.

```

14. Permission Classes

File: apps/core/permissions.py
 Role source: apps.common.choices.role.Role
 All classes inherit from rest_framework.permissions.BaseP
 All denials are logged at WARNING level.

```

from apps.core.permissions import (
    IsAdmin,
    IsAdminOrReadOnly,
    IsCustomer,
    IsVerified,
    IsNotAuthenticated,
    RoleBasedProfilePermission,
)

```

Permission Class	Rule
IsAdmin	user.role == Role.ADMIN
IsAdminOrReadOnly	GET/HEAD/OPTIONS: anyone

	POST/PUT/PATCH/DELETE: A
IsCustomer	user.role == Role.CUSTOM
IsVerified	user.is_verified == True
IsNotAuthenticated	not user.is_authenticate
RoleBasedProfilePermission	Verified CUSTOMER or ADM

Usage in Views:

```
class ProductListAPIView(BaseAPIView):
    permission_classes = [IsAdminOrReadOnly]

class CartDetailAPIView(BaseAPIView):
    permission_classes = [IsAuthenticated, IsVerified]
```

NOTE: BaseAPIView defaults to permission_classes = [AllowAllMethodPermission]
Always override explicitly for protected endpoints.

15. Middleware

File: apps/core/middleware.py

Position in MIDDLEWARE list: 2nd (after CorsMiddleware)

RequestIDMiddleware

```
class RequestIDMiddleware:
    def __call__(self, request):
        request.id = str(uuid.uuid4()) # unique id
        response = self.get_response(request)
        response["X-Request-ID"] = request.id # also store in response
        return response
```

Every request has request.id.

BaseAPIView.transform_payload() reads this and injects in payload
Exception handler reads getattr(request, "id", None) – same as request.id

16. Authentication and Token Strategy

JWT Configuration (SimpleJWT)

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME':    timedelta(minutes=1),
    'REFRESH_TOKEN_LIFETIME':   timedelta(days=1),
    'ROTATE_REFRESH_TOKENS':    True,           # new refresh token
    'BLACKLIST_AFTER_ROTATION': True,           # old refresh token
    'AUTH_HEADER_TYPES':        ('Bearer',),
}
```

Token Storage Strategy

Token	Location	Rationale
Access Token	Response body data	Frontend stores it NEVER local storage
Refresh Token	HttpOnly cookie	JavaScript cannot access it Browser sends it

Cookie Settings for Refresh Token

```
Name:      refresh_token
HttpOnly:  true
Secure:    true (production) / false (local dev)
SameSite:  Lax
MaxAge:    7 days
Path:      /api/accounts/
```

Cookie scoped to /api/accounts/ – NOT sent on production

CSRF Strategy

```
CSRF Token: Set as readable cookie
Frontend:   Reads cookie, sends in X-CSRFToken header
Pattern:    Double-submit CSRF protection
```

Auth Flow for Protected Endpoints

1. Frontend sends Authorization: Bearer <access_token>

2. DRF JWTAuthentication validates the token
3. If expired → frontend uses refresh token (auto-sent)
POST /api/accounts/token/refresh/ for a new access
4. Frontend retries original request with new access

17. Logging System

Logger Names – One Per App

```
import logging
logger = logging.getLogger("apps.accounts")    # in accounts app
logger = logging.getLogger("apps.products")    # in products app
logger = logging.getLogger("apps.orders")      # in orders app
logger = logging.getLogger("apps.cart")        # in cart app
logger = logging.getLogger("apps.core")        # in core app
```

Use the correct logger for the correct app.

Do NOT use the root logger – it has level WARNING, DEPRECATED

Log Files

Logger	File
-----	-----
apps.accounts	logs/accounts.log
apps.products	logs/products.log
apps.orders	logs/orders.log
apps.cart	logs/cart.log
apps.core	logs/core.log
django, celery	logs/django.log

All loggers also write to console. Level is DEBUG for all

Log Format

```
[2024-01-15 14:30:22] INFO apps.products ProductListAPIView
[YYYY-MM-DD HH:MM:SS] LEVEL name message
```

Logging Policy

```

In services (raise exceptions, add context):
    logger.info("Order created: order_id=%s user_id=%s")
    logger.warning("Inventory low: variant_id=%s stock=%s")

In exception handler (automatic – do not duplicate):
    DomainError                                → WARNING, no traceback
    InfrastructureError notify=True             → ERROR, exc_info=True
    InfrastructureError notify=False           → WARNING, no traceback
    DRF 4xx                                     → WARNING, no traceback
    DRF 5xx                                     → ERROR, exc_info=True
    Unhandled                                  → ERROR, exc_info=True

NEVER do this – logging in builder functions or serializers
    logger.error("...", exc_info=True) # in _non_field_error
    # Logging belongs in the handler and service layer

```

SensitiveDataFilter

```

Applied to the file handler for django.log.
Masks sensitive fields before writing to disk.
File: apps/core/logging.py
Do not log passwords, tokens, or PII in any log call.

```

18. Settings Reference

File: config/settings/base.py (single settings file – no overrides)

Key Settings

Setting	Value
AUTH_USER_MODEL	"accounts.User"
TIME_ZONE	"UTC" (database timezone)
CELERY_TIMEZONE	"Asia/Karachi" (PKT for services)
DEFAULT_AUTO_FIELD	"django.db.models.BigAutoField"
EXCEPTION_HANDLER	"apps.core.api.exceptions.handler"
DEFAULT_SCHEMA_CLASS	"drf_spectacular.openapi.AutoSchema"

Redis port	6380 (NOT default 6379 – i
Access token lifetime	1 minute
Refresh token lifetime	1 day

Cache Backend Names (exact strings used in TwoLevelCa

caches["default"] → Redis (L2, shared across worker
caches["local"] → LocMemCache (L1, per-process)

Do not change these names without updating apps/core/

Installed Apps Load Order

1. DJANGO_APPS (django.contrib.*)
2. THIRD_PARTY_APPS (rest_framework, simplejwt, cor
3. PROJECT_APPS (apps.core, apps.common, apps.a

Default Throttle Rates

CustomAnonRateThrottle → 10000/hour
CustomUserRateThrottle → 1000/hour
Custom throttle classes in apps/core/throttles.py

Debug Toolbar

Enabled ONLY when DEBUG=True – middleware appended co
INTERNAL_IPS includes Docker internal IP detection lo

Timezone Critical Note

TIME_ZONE = "UTC" for the database.
CELERY_TIMEZONE = "Asia/Karachi" for scheduled tasks.
When bounding date ranges in nightly analytics aggreg
use timezone.localtime() to avoid midnight bleeding a
Failure to do this produces daily snapshots that span

19. Critical Rules – Never Violate

These are enforced by architecture. Violating them causes hard to trace and inconsistencies that break the frontend

Response Shape Rules

1. Every view MUST inherit from BaseAPIView – never A
2. Never construct a raw Response() in a view.
Always use the helper methods: success_response(),
3. Never put data in the errors field or errors in th
4. meta must always contain request_id.
This is automatic via transform_payload(). Do not

Exception Rules

5. Raise DomainError and InfrastructureError in the s
Let custom_exception_handler convert them to respo
Do NOT catch and re-raise as DRF APIException – yo
6. Never pass exc.internal to any builder function.
It must only appear in log calls inside custom_exc
7. Never log inside builder functions (_non_fields_do
ONE log entry per exception. Logging belongs in cu
8. DomainError status_code MUST be 400 or 409 only.
The constructor raises ValueError if you pass anyt
9. InfrastructureError is always 503. You cannot chan
If you need a different 5xx, use DRF APIException

Error Code Rules

10. Never use string literals at raise sites.
Always import and use ErrorCode constants from ap
11. Never DELETE an ErrorCode – deprecate with a comm
The frontend switches on these values. Deleting b

Cache Rules

12. Never call KEYS * on Redis in any context.
Always use delete_pattern() which uses SCAN internally.
13. Never cache user-specific sensitive data in share without scoping the cache key to the user ID.
14. Never convert Decimal monetary values to float due to precision loss.
Use decimal.Decimal and Django Sum() with DecimalField.

Inventory and Transaction Rules

15. Wrap checkout logic in transaction.atomic().
Use ProductVariant.objects.select_for_update() without this, concurrent checkouts WILL oversell.
16. Never mutate order status without writing an OrderStatusChangeLog.
Use a service method that atomically updates both order and status.

Logging Rules

17. Use the correct logger for the correct app.
logger = logging.getLogger("apps.products") in products app
logger = logging.getLogger("apps.orders") in orders app
18. Never use the root logger – logging.getLogger() without specifying a module.
Root logger level is WARNING. DEBUG/INFO messages are ignored.
19. Celery tasks that call external APIs MUST use exponential backoff.
Never run synchronous external HTTP calls in the Celery task.

Timezone Rules

20. TIME_ZONE = "UTC" for the database.
CELERY_TIMEZONE = "Asia/Karachi" for scheduled tasks.
Always use timezone.localtime() when bounding dates for reports.
nightly analytics – never assume UTC midnight = PM 12:00.

20. How to Add a New Endpoint – Checklist

Follow this every time. Skip no step.

Step 1 – Models

- [] Model defined in `apps/{app}/models.py`
- [] Migration created and applied
- [] `__str__` defined on every model
- [] Correct `on_delete`: PROTECT for financial records,
- [] `BigAutoField` as default (set globally – no action

Step 2 – Error Codes

- [] Any new error codes added to `apps/core/error_code`
- [] Grouped with a comment indicating which app they
- [] Frontend team notified of new codes

Step 3 – Exceptions (if needed)

- [] Service raises `DomainError` for business rule viol
- [] Service raises `InfrastructureError` for external s
- [] `exc.internal` contains ONLY diagnostic data (never
- [] `exc.client_extra` contains ONLY data safe to expos

Step 4 – Service Layer

- [] Business logic in `apps/{app}/services.py` – never
- [] DB queries via `selector/repository` functions – ne
- [] `transaction.atomic()` wrapping any multi-step writ
- [] `select_for_update()` on any `inventory/stock` rows b

Step 5 – Serializer

- [] Serializer in `apps/{app}/serializers.py`
- [] Timestamp fields use `TimestampFieldsMixin` or Cust
- [] No business logic in serializers – validation onl
- [] `error_response(errors=serializer.errors)` for vali

Step 6 – View

```
[ ] View inherits from BaseAPIView – never APIView or
[ ] permission_classes set explicitly (do NOT rely on
[ ] Paginated lists use get_pagination_params() + bui
[ ] Cache via two_level_cache.get_or_set() for read-h
[ ] Correct response helper used: success_response /
[ ] Logger declared: logger = logging.getLogger("apps
[ ] Logger calls present for key actions: create, upd
```

Step 7 – URLs

```
[ ] URL added to apps/{app}/urls.py with app_name set
[ ] Named with descriptive name: name="order-detail"
[ ] Registered in config/urls.py under correct API pr
```

Step 8 – Testing (Postman)

```
[ ] Full envelope shape verified: success, message, d
[ ] request_id appears in meta
[ ] errors.code matches expected value for error scen
[ ] errors.non_fields.category is correct: domain / s
[ ] exc.internal does NOT appear in response body
[ ] Pagination: page, page_size, showing_from, showin
[ ] Cache: second request shows source="l2_redis" or
```

21. Appendix A – Response Examples by Scenario

200 – Paginated List

```
{
    "success": true,
    "message": "Products retrieved successfully.",
    "data": [
        {"id": 1, "name": "Chain Sprocket", "price":
    ],
    "errors": null,
```

```
    "meta": {
      "request_id": "abc123",
      "page": 1,
      "page_size": 12,
      "total": 54,
      "total_pages": 5,
      "showing_from": 1,
      "showing_to": 12,
      "has_next": true,
      "has_previous": false,
      "source": "l2_redis",
      "sort": "newest"
    }
  }
```

201 – Resource Created

```
{
  "success": true,
  "message": "Order placed successfully.",
  "data": {
    "order_id": "ORD-2024-00123",
    "total": "4500.00"
  },
  "errors": null,
  "meta": {"request_id": "abc123"}
}
```

400 – Field Validation Failure

```
{
  "success": false,
  "message": "Registration failed.",
  "data": null,
  "errors": {
    "code": "validation_error",
    "fields": {
      "email": {
        "message": "Enter a valid email address",
        "code": "invalid_email_format"
      }
    }
  }
}
```

```
        }
      },
      "non_fields": null
    },
    "meta": {"request_id": "abc123"}
  }
}
```

400 – Domain Rejection

```
{
  "success": false,
  "message": "Your cart is empty.",
  "data": null,
  "errors": {
    "code": "validation_error",
    "fields": null,
    "non_fields": {
      "category": "domain",
      "message": "Your cart is empty. Add item",
      "code": "cart_empty",
      "extra": null
    }
  },
  "meta": {"request_id": "abc123"}
}
```

401 – Authentication Required

```
{
  "success": false,
  "message": "Authentication required.",
  "data": null,
  "errors": {
    "code": "authentication_error",
    "fields": null,
    "non_fields": {
      "category": "validation",
      "message": "Authentication credentials w",
      "code": "authentication_error",
      "extra": null
    }
  }
}
```

```
    }
  },
  "meta": {"request_id": "abc123"}
}
```

409 – State Conflict

```
{
  "success": false,
  "message": "This order has already been cancelled",
  "data": null,
  "errors": {
    "code": "conflict_error",
    "fields": null,
    "non_fields": {
      "category": "domain",
      "message": "This order has already been",
      "code": "order_already_cancelled",
      "extra": null
    }
  },
  "meta": {"request_id": "abc123"}
}
```

503 – Infrastructure Failure

```
{
  "success": false,
  "message": "The service is temporarily unavailable",
  "data": null,
  "errors": {
    "code": "server_error",
    "fields": null,
    "non_fields": {
      "category": "system",
      "message": "Payment gateway is temporarily",
      "code": "payment_gateway_timeout",
      "extra": {"retry_after": 30}
    }
  },
}
```

```
    "meta": {"request_id": "abc123"}
}
```

22. Appendix B – Import Reference

Core infrastructure – use these, never reinvent

```
from apps.core.api.views          import BaseAPIView
from apps.core.api.exceptions import (
    _format_drf_errors,
    _non_fields_domain,
    _status_to_error_code,
    register_monitor,
)
from apps.core.exceptions          import DomainError, Inf
from apps.core.error_codes         import ErrorCode
from apps.core.cache               import two_level_cache
from apps.core.paginations         import get_pagination_p
from apps.core.fields              import CustomDateTimeFi
from apps.core.mixins              import TimestampFieldsM
from apps.core.permissions         import (
    IsAdmin,
    IsAdminOrReadOnly,
    IsCustomer,
    IsVerified,
    IsNotAuthenticated,
    RoleBasedProfilePermission,
)
```

Document version: 1.0

Last updated: Based on actual source files read in full.
Do not modify this document without reading the actual so
Do not guess at implementation details – ask for the file
``